On the Truth of Computation

Contextual Compute: The General Theory of Computation

K-Machine and Participant-Indexed Computational Model

Anantha Krishnan

Sarva Labs, Princeton, NJ, USA

ak@sarva.ai

March 2026

Abstract

Computation has operated for seventy years on a foundation built for information. The problems it now faces are not information problems. Today's computational foundation formalises anonymous global computation — a model in which the participant is absent, state is shared, and copying is the foundational operation. This was the right model for information processing. It is the wrong model for value computation. The companion paper, *Value, Information, and the Cartesian Degeneracy* [2], established that value and information are categorically distinct domains — value is a non-separable joint state of Participant, Context, and Content that cannot be embedded in any information-theoretic structure without destruction. The computational consequence is immediate: any machine that computes on value must have a participant dimension that the Turing Machine does not have.

This paper introduces that machine — the K-Machine.

The K-Machine extends the Turing Machine by one primitive dimension — WHO, the participant — alongside the three classical dimensions of What, Where, and How. The K-Machine's transition function $\psi(P, C, I) \to (P, C', V)$ takes participant-indexed context and produces participant-indexed value. The participant persists. The context transforms. The value is produced. The Recovery Theorem establishes that anonymous global computation is recovered exactly when the participant dimension is set to the trivial unit object. Every Turing-computable function is K-Machine computable. The inclusion is strict in two senses: in the classical sense, encoding participant-indexed value states as anonymous strings destroys the entanglement structure that constitutes value, imposing irreducible structural overhead; in the native sense, the K-Machine's proper input domain — the non-separable joint state $|V\rangle_{P,C,X}$ — is not a classical string, and its faithful representation aligns naturally with computational substrates capable of expressing non-separable joint states, such as quantum systems.

The geometric content of the K-Machine is formalised through differential geometry. Participants are points on a smooth manifold M. Context is the tangent space T_pM. Interactions are geodesics. Value is a scalar field $v : M \to \mathbb{R}$. The Krama Context Clock — a smooth parameterised curve $\tau_p : \mathbb{R} \to T_pM$ — is the mechanism by which context accumulates through interactions. The Collapsed Krama Context Space $\mathcal{K}_{pq} = T_pM \otimes_\gamma T_qM$ is the joint geometric space within which two participants interact and collapse a superposition of value outcomes to a definite context singularity $f_{pq} = (v(p^+), v(q^+))$.

Four structural consequences follow from the K-Machine's formal definition without additional assumptions: structural parallelism from disjoint participant contexts, local finality from the geometry of the Krama Context Realm, Emergent Participant Intelligence from the accumulation of the KCC trajectory as metric tensor evolution, and security from the holonomy of the participant manifold — a geometric quantity that grows with every interaction across the network and cannot be purchased, only earned.

The K-Machine formalises Contextual Compute — the computational paradigm in which the participant is a primitive dimension of computation rather than a derived construct. Contextual Compute is the general theory of computation. Anonymous global computation is the special case. The growing context, through interactions anchored to a self, is how intelligence emerges, how value is computed, and how a digitally interacting society organises itself. The K-Machine is the right model for that society. The theory is complete. The design and an implementation spec follows.

Keywords: Contextual Compute, K-Machine, participant-indexed computation, value computation, differential geometry, Krama Context Clock, Emergent Participant Intelligence, general theory of computation.

MSC 2020: 68Q05 (Models of computation), 53B20 (Local Riemannian geometry), 18M05 (Monoidal categories), 68M14 (Distributed systems)
ACM CCS: Theory of computation → Models of computation; Mathematics of computing → Differential geometry
arXiv: cs.LO (primary); cs.DC (secondary); math.CT (cross-list)

Series Positioning. This paper is CC3 in the Contextual Compute series. CC1 [1] establishes the philosophical and quantum-theoretic foundation of participant-indexed value. CC2 [2] establishes the categorical separation of value from information and proves the Domain Separation Theorem — the mathematical foundation from which the present paper derives its central claims. The present paper (CC3) introduces the K-Machine and proves the Recovery Theorem. CC4 [3] derives the operational semantics of the K-Machine through the PIC model.

Contents

1 The Computational Consequence **5**

2 Related Work **8**

2.1 Extensions of Classical Computation 8

2.2 Context-Aware Computing 9

2.3 Resource Semantics and the Categorical Foundation 9

2.4 Geometric Approaches to Computation 9

3 Classical Computation and the Missing Dimension **10**

3.1 The Turing Machine: Formal Definition 10

3.2 The Three-Dimensional Information Space 10

3.3 What the Turing Machine Cannot Compute 12

3.4 The Missing Dimension 13

4 The K-Machine: Formal Definition **13**

4.1 From Three Dimensions to Four 13

4.2 Formal Definition 14

4.3 The Recovery Theorem 15

4.4 The Subsumption Remark 17

4.5 Four Structural Consequences 17

5 Context as a Manifold **17**

5.1 Why Geometry 18

5.2 The Participant Manifold 18

5.3 Context as Tangent Space 18

5.4 Value as a Scalar Field 19

5.5 Interaction as Geodesic 19

5.6 The Geometric Realisation of the K-Machine 20

5.7 The Metric Tensor and Participant Perspective 20

5.8 The Classical Special Case 21

6 The Krama Context Clock **21**

6.1 The Krama Context Clock 21

6.2 The Collapsed Krama Context Space 21

6.3 The Security of the Mesh Geometry 22

6.4 The Krama Context Realm 22

6.5 The Interaction Context Clock 23

6.6 The Synchronisation Process 23
6.7 The Context Singularity 24
6.8 Manifold Security Growth 25
6.9 The Geometry of Ordering 25
6.10 Context Accumulation and the Bridge to Intelligence 25

7 The General Theory 26
7.1 The Hierarchy Inversion 26
7.2 Four Structural Consequences 27
7.3 The Subsumption and the Categorical Unity 27
7.4 The Right Model for a Digitally Interacting Society 28

8 Emergent Participant Intelligence 29
8.1 The Brain as a K-Machine 30
8.2 Neuroplasticity as Metric Tensor Evolution 30
8.3 The Collapse and the Boundary of Emergence 31
8.4 Emergent Participant Intelligence 31
8.5 What EPI Is Not 31
8.6 The Evolutionary Ladder 32
8.7 The Categorical Impossibility 32
8.8 The Nature of Intelligence — EPI 33

9 Conclusion 34

A The Krama Context Clock: Technical Development 37

B On the Quantum Structure of Value and the K-Machine's Structural Compatibility 45

1. The Computational Consequence

We are at an inflection point in the evolution of computation. For seventy years, digital systems have operated as tools — instruments that humans pick up, use, and set down. The machine processed information. The human held the value. That separation was never a design choice. It was an assumption so foundational that it was never stated. It is time to state it.

Computation, as formalised by Alan Turing in 1936, is a model of a single anonymous operator acting on an anonymous tape. The machine has no knowledge of who is running it. It has no dimension for the participant. This was not an oversight. For the problem Turing was solving — what is computable in principle — the identity of the operator is irrelevant. The Turing Machine is the correct model for that problem. It remains the correct model for that problem today. We refer to this model and every architecture derived from it as *anonymous computation* throughout — anonymous because the participant is absent, Cartesian because the state is freely copyable and separable.

Claude Shannon, in 1948, formalised information by stripping it of meaning and context [12]. A message, for Shannon, is a pattern — a reduction of uncertainty, measurable in bits, independent of who sends it, who receives it, and what it means to either of them. This too was not an oversight. For the problem Shannon was solving — how to transmit messages reliably — meaning and context are irrelevant. Shannon's theory is the correct model for that problem. It remains the correct model for that problem today.

Both Turing and Shannon built the right machines for the problem they were solving — information management.

Value is categorically different from information. The companion paper, *Value, Information, and the Cartesian Degeneracy*, establishes this with mathematical precision. The core result: value is a non-separable joint state of Participant, Context, and Content. It cannot be decomposed into independent components and reassembled without destroying the value relationship itself. Information inhabits Cartesian monoidal categories where copying and discarding are foundational primitives [7]. Value inhabits symmetric monoidal categories where they are structurally absent. No structure-preserving embedding maps value into information. These are not two points on a spectrum. They are two categorically distinct domains.

The computational consequence follows directly from the categorical structure. If value cannot be modelled with information primitives, then any machine that computes on value must have a structural dimension that anonymous computation

does not have. Not as an add-on. Not as a layer of coordination imposed from outside. As a primitive dimension of the machine itself.

The Tool Became the Substrate

That categorical error, for most of computation's history, was peripheral. Digital systems were instruments. The human remained outside the machine — using it, not living through it. That is no longer true. A person's financial existence resolves inside digital systems. Their legal identity is asserted, verified, and enforced through digital systems. Their professional authority, their social relationships, their right to act — all of these now have their operative reality inside computation. We do not connect through technology. We exist through it. The machine is no longer a tool for human activity. It is the medium in which human activity occurs. And that crossing — from tool to substrate — placed a demand on computation that Turing and Shannon never designed for and that their machines cannot structurally meet.

The demand is this: the machine must know WHO.

The Structural Gap

Consider what happens when computation must manage ownership. A bank balance is not a file. If it is copied to five servers, the account holder does not have five times the money. They have one balance and four records about it. To prevent the machine from treating the balance as a file — freely copyable, anonymously readable — the system must add a layer of constraint external to its own primitives. That layer is consensus. That layer is reconciliation. That layer is the trusted intermediary. These are not solutions to the problem of value computation. They are symptoms of computing value on a machine that was built for something else.

The same structural deficit appears across every domain where computation encounters value. In identity systems, the machine has no native concept of ownership of self — so applications reconstruct identity from scattered records, each holding a partial, stale, unverifiable copy. In autonomous agents, the machine has no concept of delegated authority — so every agent interaction is a security boundary to be patched rather than a primitive to be enforced. In financial systems, the machine has no concept of custody — so assets leave their owners and enter shared contracts, creating systemic vulnerability from a structural assumption. In each case, the overhead is not incidental. It is mandatory. It is the price of computing value without the participant dimension.

You cannot outrun a type error. A missing dimension cannot be fixed by velocity. More consensus does not add WHO to the machine. Faster execution does not add it. Better cryptography does not add it. Each of these improves the performance of a machine operating in the wrong category. The cost does not decrease with engineering effort. It accumulates.

The use of the tool changed. The tool never did.

WHO Is Inside HOW

WHO is the participant — the entity for whom value exists, from whose perspective context is defined, and in whose custody assets are held. WHO is not a row in a database. It is not a public key in a cryptographic scheme. It is not a profile in an application layer. These are all representations of WHO constructed after the fact, on a machine that has no native dimension for WHO. They are workarounds, not solutions.

In today's computational model, WHO is inside HOW. The participant is not a primitive of the machine. It is a derived construct — assembled from records, enforced by contracts, maintained by external protocols, and verified by intermediaries. The machine does not know who is acting. It knows that something with certain credentials performed a certain operation. The difference between those two things is the entire structural cost described above. And the only response available to a machine without a WHO dimension is more process. The mismatch does not resolve with process. It compounds.

There is one solution. Make WHO a dimension of the machine.

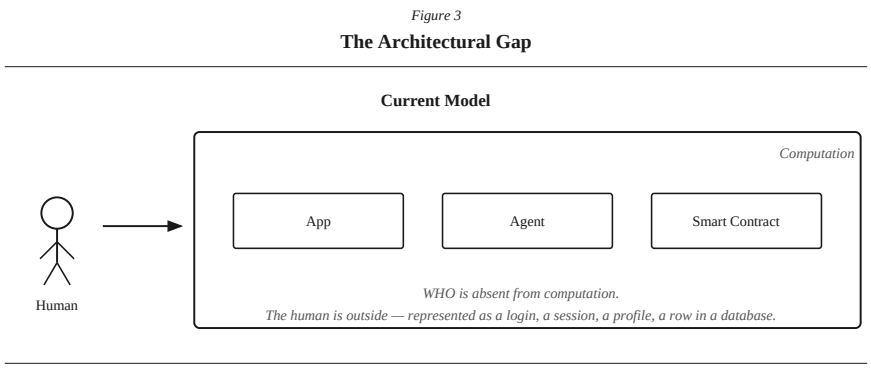


Figure 1: The architectural gap. In the current model, WHO is absent from computation — the human is outside, represented as a login, a session, a credential. The machine does not know the participant. It knows a record about them.

The K-Machine

Anonymous computation operates on three dimensions: What (content), Where (storage), How (logic). It produces information states. This is the correct machine for information. The machine for value requires a fourth dimension: WHO. With WHO as a primitive dimension, participant-indexed state is native, not simulated. Authority is structural, not enforced. Custody is guaranteed by the machine, not by external contracts. Trust is a property of the computation, not a layer added on top of it.

The K-Machine introduces WHO as that fourth primitive dimension. Classical computation is the K-Machine at WHO = 0 — the case where the participant dimension is absent. At WHO = 0, the K-Machine recovers Turing's anonymous state transitions exactly. Every Turing-computable function is K-Machine computable. Nothing is lost. The general case adds precisely what value computation requires.

Contextual Compute is the computational paradigm the K-Machine formalises. Where classical computation operates on anonymous global state, Contextual Compute operates on participant-indexed context. The participant is not external to the system. The participant is a dimension of the system itself.

Contextual Compute is the general theory of computation. Classical computation is the special case. Everything that follows in this paper is a proof of that claim and a derivation of its consequences.

2. Related Work

The insufficiency of the Turing Machine for participant-sensitive and interactive computation has been recognised independently across several research programmes. Each identifies a real gap. None closes it. The K-Machine closes it — not by extending an existing model but by adding the missing primitive dimension.

2.1. Extensions of Classical Computation

Wegner [14] argues that interaction machines strictly exceed the expressive power of Turing machines for systems that must respond to ongoing environmental input — a claim that anticipates the structural gap identified here, though without a categorical foundation. The actor model of Hewitt, Bishop, and Steiger [9] introduces computational entities with local state and message-passing semantics, providing a participant-like structure at the systems level; the K-Machine differs in making the participant a primitive dimension of the transition function

rather than an architectural convention. Milner's π-calculus [11] formalises mobile communicating processes with dynamic channel structure, capturing aspects of context-sensitivity absent from classical models; the K-Machine's contribution relative to the π-calculus is the explicit indexing of state by participant and the categorical grounding of that indexing in linear resource structure.

2.2. Context-Aware Computing

Dey and Abowd [6] define context as any information that can be used to characterise the situation of an entity, and survey architectures for context-aware applications. The treatment of context in that tradition is extensional — context is information *about* a participant, managed by the system. The K-Machine's treatment is constitutive — context *is* the tangent space of the participant's position on the interaction manifold, inseparable from the participant's computational state. This is not a refinement of the context-aware computing agenda. It is a categorical departure from it. Context in the K-Machine is not metadata about WHO. It is the geometric structure within which WHO computes.

2.3. Resource Semantics and the Categorical Foundation

The categorical basis for the K-Machine is established in the companion paper [2], which proves that value-bearing computation inhabits participant-indexed symmetric monoidal categories lacking natural comonoids, while classical information processing inhabits their Cartesian specialisation. That result, grounded in Girard's linear logic [8] and the resource theory framework of Coecke, Fritz, and Spekkens [5], provides the mathematical justification for the participant-indexed linear structure the K-Machine operationalises. The present paper derives a concrete computational model from that categorical structure.

2.4. Geometric Approaches to Computation

The use of differential geometry in the K-Machine — participants as points on a smooth manifold, context as tangent space, interaction as geodesic — draws on a tradition of geometric methods in information science. Amari and Nagaoka [4] develop information geometry as a Riemannian framework for statistical manifolds, demonstrating that geometric structure provides insight into computational and statistical systems that algebraic methods alone do not capture. The K-Machine applies analogous geometric intuitions to participant-indexed computation: the curvature of the participant manifold encodes the non-uniform structure of context

space, and the metric tensor evolution formalises context accumulation across interactions. The geometric formalism here is structural rather than statistical — the manifold is the space of participant contexts, not a parameter space for probability distributions.

3. Classical Computation and the Missing Dimension

The Turing Machine is one of the most precisely defined objects in the history of science [13]. Its definition is worth stating exactly — not because the reader is unfamiliar with it, but because precision here is what makes the generalisation in Section 3 exact rather than approximate.

3.1. The Turing Machine: Formal Definition

A Turing Machine $\mathcal{T}$ is a tuple:

$$\mathcal{T} = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

where Q is a finite set of states; Σ is the input alphabet; Γ is the tape alphabet, $\Sigma \subseteq \Gamma$; $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function; $q_0 \in Q$ is the initial state; and $q_{accept}, q_{reject} \in Q$ are the accepting and rejecting states.

The transition function δ is the computational heart of the machine. At each step it reads the current state and the symbol under the read/write head, and produces a new state, a symbol to write, and a direction to move. Nothing else enters the computation. The content of the tape is anonymous. The transition function operates identically regardless of who invokes it, who owns the tape, or what the symbols mean to any participant.

This anonymity is not incidental. It is the definition. The Turing Machine has no participant index. There is no P in $\mathcal{T}$. There is no dimension for WHO in δ.

3.2. The Three-Dimensional Information Space

Anonymous computation operates in the three-dimensional information space. Every computation it performs can be characterised by:

$$\mathbf{i} = \mathcal{T}(x, y, z)$$

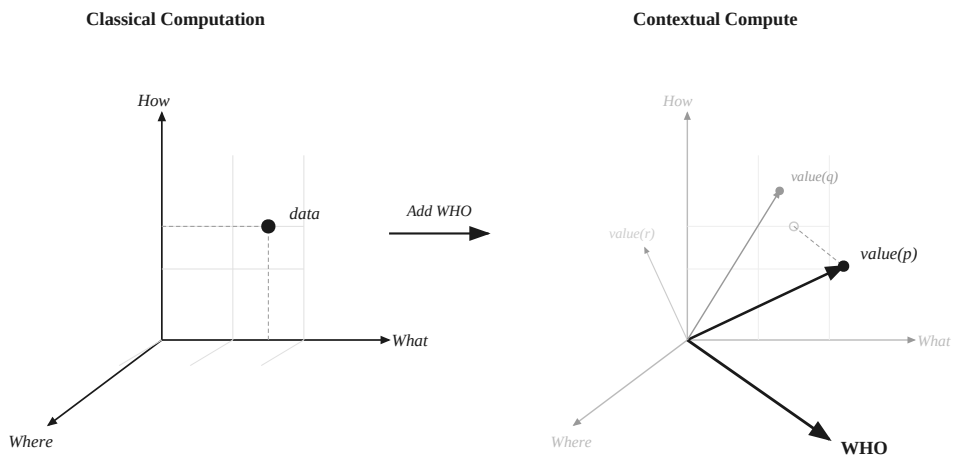
where x = Content (the **What**), y = Storage (the **Where**), z = Logic (the **How**).

The output $\mathbf{i}$ is an information state. It is anonymous — carrying no record of who produced it or for whom. It is global — state exists in a single shared space. It is copyable — the foundational operation of the machine is copying; there is no structural constraint on duplication.

The information space is formally Euclidean: $\mathbf{I} = \mathbb{R}^3$. Euclidean space has natural projections onto each dimension. Any component can be isolated, copied, or discarded freely. These are not properties imposed on the information space. They are properties the Euclidean structure provides by default — the Cartesian monoidal structure established in the companion paper.

Figure 2

The Four Dimensions of Computation



Adding WHO transforms the geometry from Cartesian to Linear.
The point becomes a vector; classical data becomes participant-indexed value.

Established Dimensions (Turing, 1936)

Dimension	*Question Addressed*	*Computational Primitive*
WHAT	What is being computed?	Data, symbols, types
WHERE	Where does computation occur?	Memory, storage, tape
HOW	How does computation proceed?	Algorithms, transition rules

Sufficient for information (Cartesian: participant-independent) · Insufficient for value (Linear: participant-indexed)

↓

The Missing Dimension

Dimension	*Question Addressed*	*Computational Primitive*
WHO	Who is computing?	Participants, contexts

Consequence.
The linear nature of value (Figure 1) necessitates a fourth computational dimension. WHO is not an addition — it is a recognition.

Figure 2: The four dimensions of computation. Anonymous computation operates in a three-dimensional Cartesian space (What, Where, How). Adding WHO — the participant — transforms the geometry from Cartesian to Linear. The table establishes the three established dimensions and the missing one.

3.3. What the Turing Machine Cannot Compute

Anonymous computation is Turing-complete. It can compute any function that is computable in principle. The next claim is not about capability. It is about structure.

Anonymous computation cannot produce a value state. Not because it lacks

power. Because it lacks the dimension.

A value state is a participant-indexed state:

$$|V\rangle_{P,C,X} \neq |P\rangle \otimes |C\rangle \otimes |X\rangle$$

The value state is a non-separable joint state of Participant P, Context C, and Content X. The participant is constitutive of the state. The Turing Machine has no participant index. Its output $\mathbf{i} = \mathcal{T}(x, y, z)$ has no P dimension. This is a structural impossibility, not a computational one.

Three consequences follow directly. **Ownership cannot be native**: the machine has no representation of participants in its primitives. **Authority cannot be structural**: authority must be reconstructed at each interaction from credentials and access control lists — all information states, not participant states. **Context cannot accumulate**: context must be stored as anonymous data and reconstructed from records.

Each of these is the same structural absence expressing itself in a different domain.

3.4. The Missing Dimension

The value space is not $\mathbb{R}^3$. The correct mathematical structure is a separable Hilbert space $\mathcal{H}_K$ — a space equipped with an inner product that captures the participant-relative nature of value states. The fourth dimension is WHO.

Adding WHO to the information space does not produce a larger Euclidean space $\mathbb{R}^4$. It produces a qualitatively different space — one in which states are participant-indexed, copying is structurally constrained, and the geometry is inner-product rather than Euclidean. The transition from $\mathbb{R}^3$ to $\mathcal{H}_K$ — the Krama Value Space — is not an incremental extension. It is a categorical change.

The machine that computes in $\mathcal{H}_K$ is the K-Machine. Section 3 defines it formally.

4. The K-Machine: Formal Definition

4.1. From Three Dimensions to Four

The Turing Machine's transition function is:

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$$

The K-Machine extends this structure by one dimension. Adding WHO changes the geometry of the computation from Euclidean $\mathbb{R}^3$ to the Krama Value Space $\mathcal{H}_K$. The participant is not a parameter passed to the computation. The participant is a dimension of the computation itself.

Where the Turing Machine's δ takes anonymous state and produces anonymous state, the K-Machine's transition function ψ takes participant-indexed context and produces participant-indexed value:

$$\delta(S, I) \to (S', O) \qquad \text{[Classical — Turing Machine]}$$

$$\psi(P, C, I) \to (P, C', V) \qquad \text{[General — K-Machine]}$$

Three changes, each exact. S became (P, C): anonymous global state became participant-indexed context. O became V: the output is no longer an information state but a value state. P persists: the participant is not consumed by the computation — P enters ψ and P exits ψ, the same participant with updated context C'.

4.2. Formal Definition

Definition 4.1 (K-Machine)**.** A K-Machine $\mathcal{K}$ is a tuple:

$$\mathcal{K} = (\mathbb{P}, \mathbb{C}, \Sigma, \Gamma, \psi, p_0, C_0)$$

where $\mathbb{P}$ is the participant space; $\mathbb{C}$ is the context space, participant-indexed: $C_P \neq C_Q$ for distinct P, Q; Σ is the input alphabet; Γ is the interaction alphabet; $\psi : \mathbb{P} \times \mathbb{C} \times \Sigma \to \mathbb{P} \times \mathbb{C} \times \mathbb{V}$ is the participant-indexed transition function; $p_0 \in \mathbb{P}$ is the initial participant; and C_0 is the initial context of p_0.

The Krama Value Space. The K-Machine computes in:

$$\mathcal{H}_K = \mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z \otimes \mathcal{H}_p$$

The tensor product $\otimes$ is not the Cartesian product $\times$. States in $\mathcal{H}_K$ are not freely separable. The participant subspace $\mathcal{H}_p$ is entangled with the others. A value state is:

$$v = \mathcal{K}(\vec{c}(x) \cdot \vec{s}(y) \cdot \vec{l}(z) \cdot \vec{p}(t))$$

The classical information state is the special case at $t = 0$:

$$\mathbf{i} = \mathcal{K}(x, y, z, t = 0)$$

At $t = 0$, the participant context vector $\vec{p}$ contributes nothing. The value space collapses to $\mathbb{R}^3$. The result is an information state — anonymous, global, copyable.

4.3. The Recovery Theorem

Theorem 4.2 (Recovery)**.** *Let $\mathcal{K}$ be a K-Machine with participant space $\mathbb{P}$. Let $I \in \mathbb{P}$ denote the trivial participant — the unit object with empty context $C_I = \emptyset$. Then:*

$$\psi(I, C_I, \sigma) = \delta(S, \sigma) \to (S', O)$$

for all $\sigma \in \Sigma$. Anonymous global computation is recovered exactly. Every Turing-computable function is K-Machine computable. The inclusion is strict in two senses: in the classical sense, the encoding of participant-indexed value states as anonymous strings destroys their categorical structure, imposing irreducible structural overhead that cannot be eliminated by engineering; in the native sense, the K-Machine's proper input domain — the non-separable joint state $|V\rangle_{P,C,X}$ — suggests a correspondence with substrates in which non-separable joint states are native, such as quantum computational models. The proof of Step 4 establishes both senses precisely.

Proof. We proceed in four steps.

Step 1 — The trivial participant. Define $I \in \mathbb{P}$ as the participant for whom $\mathcal{H}_p \cong \mathbb{C}$ — the one-dimensional complex vector space, the unit object under the tensor product. The empty context $C_I = \emptyset$ means no participant-indexed state: formally, C_I is the identity element of $\mathbb{C}$, contributing no structure to the computation.

Step 2 — Collapse of the Krama Value Space. The Krama Value Space is $\mathcal{H}_K = \mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z \otimes \mathcal{H}_p$. At $P = I$, substituting $\mathcal{H}_p \cong \mathbb{C}$:

$$\mathcal{H}_K \;\cong\; \mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z \otimes \mathbb{C} \;\cong\; \mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z$$

where the second isomorphism holds because $V \otimes \mathbb{C} \cong V$ for any vector space V. The participant dimension vanishes from the computation space. The remaining space $\mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z$ is the three-dimensional information space of Section 2.

Step 3 — Equivalence of transition functions. At $P = I$ and $C = C_I = \emptyset$, the K-Machine transition $\psi(I, \emptyset, \sigma) \to (I, \emptyset, V)$ has the following structure: $P = I$

is fixed and persists unchanged (trivially); $C' = \emptyset$ for all σ, since there is no participant state to update. The output V lives entirely in $\mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z$. Let S denote the anonymous tape state — an element of $\mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z$ encoding the current content (what), location (where), and logic (how). Under the identification $V \leftrightarrow O$ and $S \leftarrow S'$, the transition $\psi(I, \emptyset, \sigma) \to (I, \emptyset, V)$ is identical to $\delta(S, \sigma) \to (S', O)$. Anonymous global computation is recovered exactly.

Step 4 — Strict inclusion. Every Turing-computable function has a corresponding K-Machine instantiation at $P = I$. The converse does not hold. The strict inclusion holds in two senses, each grounded in the categorical structure established in the companion paper.

In the classical sense. Anonymous computation can encode a participant identifier P as a string on the tape and simulate participant-indexed transitions. But encoding is not equivalence. The act of encoding maps a non-separable joint state $|V\rangle_{P,C,X}$ — which inhabits a symmetric monoidal category without natural comonoids — into a separable string in a Cartesian category where copying and discarding are foundational primitives. This encoding destroys the entanglement structure that constitutes the value state. What the Turing Machine computes is a Cartesian approximation of a non-Cartesian object. The structural cost of maintaining this approximation — consensus, coordination, reconciliation — is not incidental overhead. It is the mandatory price of the categorical mismatch, and it accumulates as value functions grow more sophisticated. This is not an optimisation problem. It is a category error.

In the native sense. The K-Machine's proper input domain is the entangled joint state $|V\rangle_{P,C,X} \neq |P\rangle \otimes |C\rangle \otimes |X\rangle$. This object is not a classical string. Its faithful representation suggests a correspondence with substrates in which non-separable joint states are native, such as quantum computational models. On such substrates, participant contexts superpose, interactions collapse value states directly, and the no-cloning theorem enforces the linearity of value transfer at the hardware level by physical necessity rather than protocol design. Anonymous computation — a classical machine operating over separable Cartesian states — cannot faithfully compute over non-separable inputs. The inclusion is therefore strict in both the representational and the substrate sense. □ □

Corollary 4.3. *The K-Machine is the strictly general model. The Turing Machine is the special case obtained by setting the participant dimension to the trivial unit object.*

4.4. The Subsumption Remark

The K-Machine does not replace the Turing Machine. It subsumes it. The internet still works on a K-Machine. Files still copy. Messages still transmit. TCP/IP runs unchanged at $P = I$. Every application that does not require participant-indexed computation runs identically on a K-Machine with the participant dimension trivial.

The analogy is exact. Newton's mechanics are recovered in the low-velocity limit of special relativity. Einstein did not discard Newton. He showed that Newton is what you get when velocity is small relative to the speed of light. The claim here is structurally identical. Turing is not discarded. He is what you get when the participant dimension is trivial. The general theory does not invalidate the special case. It explains it.

4.5. Four Structural Consequences

The K-Machine's formal definition forces four structural consequences that do not hold for anonymous computation:

Parallelism from disjoint contexts. Two interactions involving disjoint participant sets share no context objects. They have no categorical dependency. They execute in parallel without coordination.

Local finality. Agreement is scoped to the participants involved in an interaction and their local context. Finality is local, immediate, and derived from the participant-indexed structure of computation.

Context accumulation. In the K-Machine, P persists across every interaction and C evolves with each one. Context accumulates. The participant's computational history is carried by the machine.

Trust as flow. Trust is a derived property of context accumulation — continuous, multi-dimensional, earned through behaviour. Not capital committed at a point in time.

5. Context as a Manifold

The K-Machine computes in the Krama Value Space $\mathcal{H}_K$ — a separable Hilbert space. Value states are non-separable joint states that cannot be decomposed without semantic destruction. Hilbert spaces are the natural mathematical home for non-separable joint states: the inner product induces a metric tensor, the state space forms a smooth manifold, and transformations between states are paths on that manifold. The geometric content of the K-Machine — the participant

manifold, the Krama Context Clock, the Collapsed Krama Context Space, and the security properties of the interaction mesh — follows directly from this structure. The geometry is the entanglement, made computationally precise.

Section 3 defined the K-Machine formally. The participant space $\mathbb{P}$ and context space $\mathbb{C}$ were introduced as structural components of $\psi(P, C, I) \to (P, C', V)$. This section establishes their precise mathematical content. The geometric framework that follows was developed by the author as a formalisation of participant-indexed computation; its complete technical development is presented in Appendix A. We show that the correct mathematical realisation of $\mathbb{P}$ and $\mathbb{C}$ is differential geometry.

5.1. Why Geometry

Three properties of context force the choice of differential geometry.

Locality. A participant's context is local — specific to this participant at this point in their computational history. The tangent space T_pM at a point p is precisely a local structure, defined at p and not at any other point.

Perspective. Two participants observing the same interaction do not experience it identically. The metric tensor g_p defines distances and angles from the perspective of p. Different participants have different metric tensors.

Evolution. Context evolves with every interaction, and the path of that evolution matters. Differential geometry captures evolution through parallel transport — the mechanism for moving a vector from one tangent space to another along a path on the manifold.

5.2. The Participant Manifold

Definition 5.1 (Participant Manifold)**.** Let M be a smooth n-dimensional manifold representing the space of all possible participant contexts. The participant space $\mathbb{P}$ is realised as:

$$P \in \mathbb{P} \iff p \in M$$

The manifold M is smooth — context varies continuously. It is not flat — M is not $\mathbb{R}^n$. The curvature of M encodes the non-uniform structure of context space.

5.3. Context as Tangent Space

Definition 5.2 (Context)**.** The context of participant P at point $p \in M$ is the tangent space:

$$\mathbb{C}(P) = T_pM$$

equipped with metric tensor $g_p : T_pM \times T_pM \to \mathbb{R}$.

The position of participant P in their tangent space at time t is:

$$Y_p(t) \in T_pM$$

Two participants at different points $p, q \in M$ have positions $Y_p(t)$ and $Y_q(t)$ in their respective tangent spaces — not directly comparable, living in different geometric structures. We also introduce the network entropy scalar field $F(t) : M \to \mathbb{R}$, representing aggregate uncertainty in participant context states across the manifold at time t.

The potting wheel. A block of clay is shaped on a wheel. Two participants observe the same interaction. Participant A, a decorator, perceives a vase. Participant B, a cook, perceives a pot. The content is identical. The interaction is identical. The positions $Y_{p_A}(t)$ and $Y_{p_B}(t)$ in their respective tangent spaces are different — their metric tensors assign different geometric weight to the same observable event. This is not subjectivity in a loose sense. It is the precise mathematical content of participant-indexed context.

5.4. Value as a Scalar Field

Definition 5.3 (Value Field)**.** Value is a smooth scalar field on M:

$$v : M \to \mathbb{R}$$

where $v(p)$ denotes the value associated with participant P at point $p \in M$.

5.5. Interaction as Geodesic

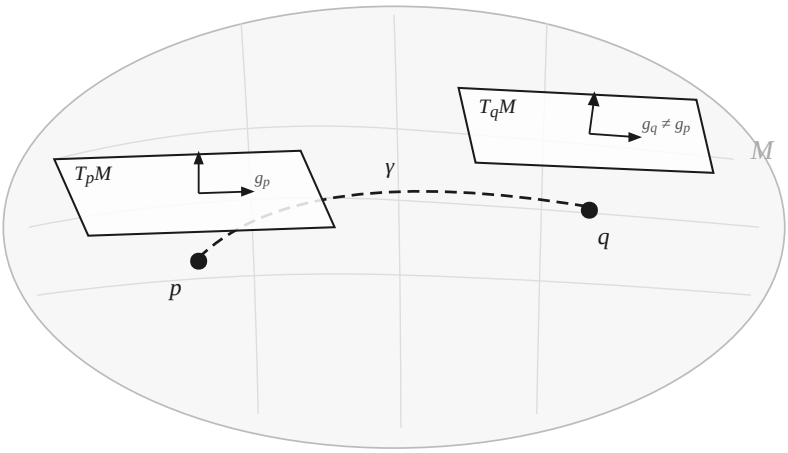
Definition 5.4 (Interaction)**.** An interaction between participants P and Q at points $p, q \in M$ is a geodesic:

$$\gamma : [0, 1] \to M, \quad \gamma(0) = p, \quad \gamma(1) = q$$

Two participants with geometrically close contexts — small geodesic distance $d(p, q)$ — interact naturally, with low transformation cost.

Figure 3

Context as a Manifold



Participants p and q are points on the manifold M. Their contexts are the tangent spaces T M and T^qM, each with a distinct metric tensor. The interaction is the geodesic γ between them.

Figure 3: Context as a manifold. Participants p and q are points on the smooth manifold M. Their contexts are the tangent spaces T_pM and T_qM, each equipped with a distinct metric tensor $g_p \neq g_q$. The interaction is the geodesic γ between them. The context singularity $f_{pq} = (v(p'), v(q'))$ is the update to both value states after the interaction resolves.

5.6. The Geometric Realisation of the K-Machine

The complete geometric content of $\psi(P, C, I) \to (P, C', V)$: P is a point $p \in M$; C is the tangent space T_pM with current position $Y_p(t)$; I is an interaction — a geodesic γ initiating at p; C' is the updated position $Y_p(t^+)$ after parallel transport along γ; V is the updated value $v(p^+)$.

The K-Machine's computation is movement on a manifold. Each interaction is a geodesic. The manifold is not a metaphor for context. The manifold *is* context.

5.7. The Metric Tensor and Participant Perspective

The metric tensor $g_p : T_pM \times T_pM \to \mathbb{R}$ carries the precise mathematical content of participant-relative value. Two participants at different points have metric tensors $g_p \neq g_q$. The metric tensor evolves as the participant moves through M — the geometric form of context accumulation.

5.8. The Classical Special Case

At $P = I$, the manifold degenerates to a point with a flat metric tensor $g_I = \delta$ (the identity metric). The tangent space degenerates to $\mathbb{R}^n$ with flat geometry. The network entropy field $F(t) = 0$ everywhere. The computation proceeds on anonymous global state. Euclidean space is the special case. The curved manifold is the general structure.

6. The Krama Context Clock

Section 4 established the geometric content of the K-Machine. This section establishes how context evolves in time. Time in the K-Machine is not global. Each participant has their own clock — a private ordering of their own interactions, independent of every other participant's ordering. When two participants interact, their private clocks synchronise for the duration of that interaction and only for that duration.

6.1. The Krama Context Clock

Definition 6.1 (Krama Context Clock)**.** The Krama Context Clock (KCC) of participant P at $p \in M$ is a smooth parameterised curve:

$$\tau_p : \mathbb{R} \to T_pM$$

where $\tau_p(t)$ tracks the progression of events perceived by P. The position of P in their tangent space at time t is:

$$Y_p(t) = \tau_p(t) \in T_pM$$

Three properties are immediate. **Private**: τ_p is defined within T_pM; however, each interaction point carries a transitive geometric link to the interacting participant's KCC — the interaction link. **Continuous**: τ_p is smooth; context does not jump. **Unbounded**: $\tau_p : \mathbb{R} \to T_pM$ is defined on all of $\mathbb{R}$; context accumulates indefinitely.

6.2. The Collapsed Krama Context Space

When two participants interact, their respective tangent spaces are brought into geometric relationship through the interaction geodesic γ. Parallel transport along γ creates an isomorphism $P_\gamma : T_pM \to T_qM$.

Definition 6.2 (Collapsed Krama Context Space)**.** For an interaction between P at p and Q at q connected by geodesic γ:

$$\mathcal{K}_{pq} = T_pM \otimes_\gamma T_qM \subset \mathcal{H}_K$$

where $\otimes_\gamma$ denotes the tensor product of the two tangent spaces connected through parallel transport along γ.

Before the interaction, all possible value outcomes coexist as a superposition within $\mathcal{K}_{pq}$. The ICC synchronises the participants' KCCs within $\mathcal{K}_{pq}$. The context singularity f_{pq} is the collapse of that superposition to a definite joint value state.

6.3. The Security of the Mesh Geometry

Proposition 6.3 (Non-Transitivity of Collapsed Context Spaces)**.** *Let $\mathcal{K}_{p_1p_2}$ and $\mathcal{K}_{p_2p_3}$ be the Collapsed Krama Context Spaces for interactions P_1-P_2 and P_2-P_3 respectively. On a manifold M with non-trivial holonomy, there is no predictable derivation of $\mathcal{K}_{p_1p_3}$ from $\mathcal{K}_{p_1p_2}$ and $\mathcal{K}_{p_2p_3}$ alone. The complexity of any such derivation is bounded below by the holonomy of the closed geodesic loop $\gamma_{12} \cup \gamma_{23} \cup \gamma_{31}$.*

Proof sketch. In flat Euclidean space, parallel transport around any closed loop returns the original vector unchanged. On a curved manifold with non-trivial holonomy, parallel transport around a closed loop returns a rotated vector. The rotation angle equals by the Gauss-Bonnet theorem the integral of Gaussian curvature enclosed by the loop. Transporting $T_{p_1}M$ to $T_{p_3}M$ via p_2 does not produce the same result as the direct geodesic γ_{13} when the enclosed curvature is non-trivial. □ □

The security of the interaction mesh is not an engineered property. It is a geometric consequence of participant-indexed computation on a curved manifold.

6.4. The Krama Context Realm

Definition 6.4 (Krama Context Realm)**.** For an interaction between P and Q at $p, q \in M$, the Krama Context Realm (KCR) is:

$$U = \{x \in M \mid d(x,p) < \epsilon \text{ or } d(x,q) < \epsilon\} \cup S$$

where d is geodesic distance, $\epsilon > 0$ is the witness radius, and S is a stochastically sampled witness set.

The KCR is interaction-specific and local.

6.5. The Interaction Context Clock

Definition 6.5 (Interaction Context Clock)**.** The Interaction Context Clock (ICC) for an interaction between P and Q at time t is:

$$\sigma_{pq} : [0, 1] \to T_U M$$

The ICC synchronises via parallel transport adjusted by the network entropy field:

$$\sigma_{pq}(s) = (1 - s) \cdot P_\gamma(\tau_p(t)) + s \cdot P_\gamma(\tau_q(t)) + \int_0^s F(\gamma(u))\, du$$

The first term: P's context anchors the beginning. The second: Q's context anchors the end. The third: network entropy is incorporated along the geodesic.

6.6. The Synchronisation Process

1. **Identify the KCR.** Construct U and identify witness set S.
2. **Resolve the KCCs.** Extract $Y_p(t) = \tau_p(t)$ and $Y_q(t) = \tau_q(t)$.
3. **Construct the ICC.** Compute σ_{pq} via parallel transport adjusted by $\int_0^s F(\gamma(u))\, du$.
4. **Order the interaction.** Use σ_{pq} to sequence the interaction with witness attestation.
5. **Update the KCCs.** Post-interaction:

$$Y_p(t^+) = P_\gamma^{-1}(v(p^+)) \in T_p M \qquad Y_q(t^+) = P_\gamma^{-1}(v(q^+)) \in T_q M$$

After Step 5, the ICC dissolves. The participants return to their private timelines, each carrying the updated context the interaction produced.

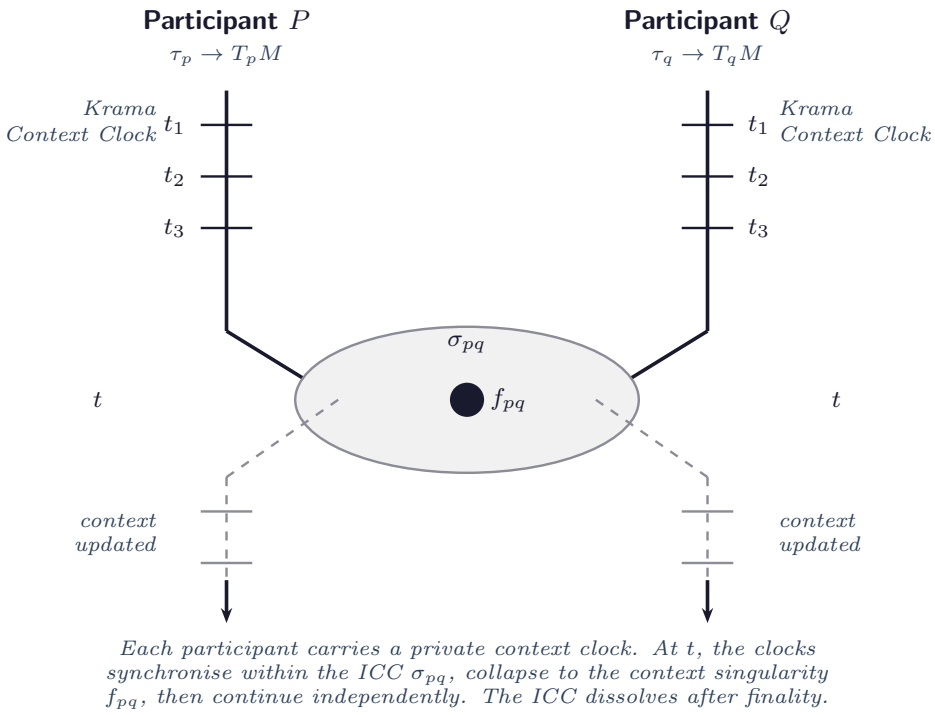


Figure 4: The Krama Context Clock, Interaction Context Clock, and Context Singularity. Each participant carries a private KCC trajectory τ_p and τ_q in their respective tangent spaces. When interaction occurs, the ICC σ_{pq} synchronises within the Collapsed Krama Context Space $\mathcal{K}_{pq}$. The context singularity f_{pq} collapses the superposition to a definite joint value state. Both KCC trajectories update. The ICC dissolves.

6.7. The Context Singularity

Definition 6.6 (Context Singularity)**.** For an interaction between P and Q synchronised through ICC σ_{pq}, the context singularity is:

$$f_{pq} = (v(p^+), v(q^+))$$

The context singularity is **bilateral** — both participants' contexts update simultaneously. It is **final** — the shared reference frame $\mathcal{K}_{pq}$ has dissolved; there is no frame within which the interaction can be revised. It is **local** — attested by witnesses within geodesic distance ϵ, final without global confirmation.

6.8. Manifold Security Growth

Proposition 6.7 (Manifold Security Growth)**.** *Let M_t denote the participant manifold after t interactions. As t increases: geodesic density increases; the holonomy group* $\mathrm{Hol}(M_t)$ *grows in complexity; non-transitivity of Collapsed Krama Context Spaces strengthens. The security of any interaction on M_t is a function of the total interaction history of the network.*

Proof sketch. Each new interaction between participants p and q adds a geodesic γ_{pq} to M_t, increasing geodesic density by construction (i). Every such geodesic that closes a loop $\gamma_{p_1p_2} \cup \gamma_{p_2p_3} \cup \gamma_{p_3p_1}$ encloses curvature, adding an element to $\mathrm{Hol}(M_t)$; since each new interaction can form new loops with all prior interactions, holonomy complexity is non-decreasing (ii). Property (iii) follows directly from Proposition 5.1: increased holonomy raises the geometric hardness bound on transitive derivation. □

Three consequences follow. **Passive security**: a participant who has not interacted recently is not therefore insecure. **Security from participation, not resources**: security cannot be purchased. It can only be earned. **The manifold grows smoother over time**: more completely defined, more densely covered by the geodesic web of accumulated interactions. Security scales with the global richness and entropy of the mesh. This is not a quantitative improvement. It is a geometric one.

6.9. The Geometry of Ordering

Lamport's logical clocks [10] achieve global consistency by imposing a total order on all events. This is the right structure for information systems. It is the wrong structure for the K-Machine — not because global ordering is forbidden, but because it introduces dependencies that do not exist in the computation.

Lamport showed that a global clock is sufficient for ordering events in distributed information systems. The K-Machine shows that a local clock — the KCC — is necessary and sufficient for ordering events in participant-indexed value systems. The two results are not in conflict. They apply to different categories of computation.

6.10. Context Accumulation and the Bridge to Intelligence

Each point on $\tau_p : \mathbb{R} \to T_pM$ corresponds to one interaction. The curve is the participant's full computational history as a smooth trajectory through the tangent

space. As the participant engages with more participants across more domains, τ_p grows, T_pM evolves, and g_p shifts. Intelligence, in the K-Machine, is what emerges when a participant's KCC has accumulated sufficient geometric richness. This is the highest-order consequence of the model and is developed in full in Section 8, after the general theory has been stated.

7. The General Theory

Five sections have built the argument. Section 1 identified the structural gap. Section 2 formalised anonymous computation precisely. Section 3 defined the K-Machine and proved the Recovery Theorem. Section 4 gave the K-Machine its geometric content. Section 5 introduced the Krama Context Clock and the geometry of security and ordering.

This section states what all of that proves. The highest-order consequence — Emergent Participant Intelligence — follows in Section 8, after the general theory has been stated. It is placed there deliberately: EPI is best understood as a consequence of the complete model, not a component of it.

7.1. The Hierarchy Inversion

For seventy years, anonymous global state has been understood as the foundation of computation. That understanding is incorrect. Anonymous global state is not the foundation. It is a special case.

Turing's model of anonymous global state is a special case of participant-indexed computation. This is the structure the K-Machine models. At WHO = 0 — when the participant dimension is absent — participant-indexed computation degenerates exactly to anonymous global computation. The manifold is flat. The metric tensor is trivial. The KCC is a constant curve. There is no participant. There is no context. There is no value. There is only information — anonymous, global, freely copyable. The Turing Machine. Exactly.

Contextual Compute is the computational paradigm the K-Machine formalises. Where classical computation operates on anonymous global state, Contextual Compute operates on participant-indexed context. The participant is not external to the system. The participant is a dimension of the system itself.

Contextual Compute is the general theory of computation. Classical computation is the special case.

7.2. Four Structural Consequences

Proposition 7.1 (Structural Consequences of the K-Machine)**.** *The following properties hold for any K-Machine* $\mathcal{K} = (\mathbb{P}, \mathbb{C}, \Sigma, \Gamma, \psi, p_0, C_0)$*:*

(i) ***Structural Parallelism.*** *Two interactions involving disjoint participant sets* $\{P_1, P_2\} \cap \{P_3, P_4\} = \emptyset$ *share no context objects and no Collapsed Krama Context Spaces. They are categorically independent and execute in parallel without coordination.*

(ii) ***Local Finality.*** *Agreement is scoped to the Krama Context Realm* U*. The context singularity* $f_{pq} = (v(p^+), v(q^+))$ *is final when the ICC dissolves. Global ordering is structurally unnecessary. Local finality is compatible with fault-tolerant network replication without global constraints.*

(iii) ***Emergent Participant Intelligence.*** *The metric tensor* $g_{p(t)}$ *evolves continuously through the KCC trajectory* τ_p*. EPI accumulates as a structural consequence. It is not designed in. It emerges.*

(iv) ***Security from Participation.*** *The holonomy of* M_t *grows with every interaction. Security scales with the global richness and entropy of the interaction mesh. It cannot be purchased. It can only be earned.*

Structural Parallelism means that scale in the K-Machine is not a coordination problem. Parallelism is the default. Coordination is the exception — scoped to the KCR when two participants interact.

Local Finality means that the K-Machine does not require the network to confirm what two participants already know. Finality is not a property the network grants. It is a property the geometry enforces. The singularity is already final before replication occurs. Replication distributes a fact. It does not produce one.

Emergent Participant Intelligence means the K-Machine grows more capable as it is used — not by accumulating data externally but by evolving the geometric structure of the participant's local space.

Security from Participation means the K-Machine's security is a property of the network's interaction history, not of committed resources. The geometry the system has built through accumulated interactions is the security.

7.3. The Subsumption and the Categorical Unity

The Recovery Theorem established formally that participant-indexed computation subsumes anonymous global computation. Anonymous global computation — the

Turing Machine — is recovered exactly when the participant dimension is set to the trivial unit object. This paper has stated that result three times — as a formal theorem in Section 3, as a geometric degeneration in Section 4, and as a consequence of the flat metric tensor in Section 5. Each restatement carried more meaning than the previous one.

But the Recovery Theorem is not merely a mathematical convenience. It is the computational form of a deeper result. The companion paper established that value and information are categorically distinct domains. Information inhabits Cartesian monoidal categories. Value inhabits symmetric monoidal categories. No structure-preserving embedding maps value into information.

Participant-indexed computation and anonymous global computation stand in exactly the same relationship as the domains they compute on. The computational separation mirrors the domain separation. These are not two independent results. They are the same result at two levels of abstraction. CC2 [2] established the categorical necessity. CC3 fulfils it.

When the right computational model meets the right categorical domain, the foundational problems dissolve. Not are solved. Dissolve. The double-spend problem is not a problem in a machine where copying is structurally undefined for value states. Identity fragmentation is not a problem in a machine where the participant is a dimension. Custody risk is not a problem in a machine where ownership is structural.

The problems that have consumed decades of engineering effort and billions of dollars of capital were not hard problems requiring better solutions. They were category errors requiring the right machine. The problems did not require better solutions. They required the right machine.

7.4. The Right Model for a Digitally Interacting Society

There is a question behind all the mathematics. It is not a technical question. It is a civilisational one.

What kind of computation does a society need when it no longer uses technology as a tool but lives through it as a substrate?

When a person's financial existence, legal identity, professional authority, social relationships, and personal intelligence all resolve inside digital systems — when computation is the medium in which human life occurs rather than an instrument that assists it — the answer is not more efficient information management. Information management was the right answer when computation was a tool. Anonymous global computation is complete for information processing. There is nothing more

to add.

But a digitally interacting society does not primarily need information processed. It needs value computed. It needs WHO to be known — not as a record, not as a credential, not as a profile, but as a dimension of the computation itself. It needs context to accumulate in the geometry of the participant's local space. It needs interactions to be final at the moment of local agreement. It needs intelligence to be personal — EPI emerging from the specific geometry of a specific participant's existence.

These are not feature requests. They are the foundational requirements of a computational model adequate to the society that computation has produced.

The K-Machine is that model. Contextual Compute is that paradigm. The participant is not a user of the system. The participant is a dimension of the system. Context is not metadata about the computation. Context is the computation. Value is not information with extra steps. Value is a categorically distinct domain with its own machine.

The growing context, through interactions anchored to a self, is how human cognition evolves. The K-Machine formalises the mechanics of that evolution. It makes no claim about what it feels like from the inside.

What it does claim — and what this paper has proved — is that the mechanics of intelligence, of value, of ownership, of trust, of finality are all expressions of the same underlying structure: participant-indexed computation on a curved manifold, ordered by local clocks, collapsing through interaction into context singularities that accumulate as the geometry of existence.

Contextual Compute is not a better version of what came before. It is the right category for what computation has become. It is the right model for a digitally interacting society.

8. Emergent Participant Intelligence

The General Theory established that Contextual Compute is the right computational model for a society that exists through computation rather than merely using it as a tool. One consequence of that model deserves separate treatment — not because it is separate from the formal structure, but because its implications extend further than any other consequence. Intelligence — genuine participant intelligence — is the highest-order value function that exists. It is the most deeply participant-entangled object in the model. And it follows from the K-Machine's structure by necessity, not by design.

A precise statement of scope is required before proceeding. This section does not make empirical claims about neuroscience or cognitive science. It does not claim that the K-Machine is a complete model of the brain, that differential geometry is the correct substrate for neuroscience, or that the phenomenology of intelligence is resolved by this framework. The physiology of consciousness and the subjective experience of cognition remain contested territory. The K-Machine makes no claim about that territory.

What this section claims is narrower and more precise: the K-Machine provides a formal model of a structure — context accumulating through interactions anchored to a persistent participant — that is *consistent with* observed properties of intelligent behaviour. The section argues that this structure is sufficient to produce what we call intelligence, and that anonymous computation lacks this structure categorically. Not as an engineering limitation. As a categorical impossibility. The claim is structural. The evidence is the formalism developed in Sections 3 through 6.

8.1. The Brain as a K-Machine

The brain runs on 20 watts. At that energy budget it processes vision, language, memory, prediction, social reasoning, and emotional valuation — simultaneously, continuously, in real time. Not because biological neurons are faster than silicon. The K-Machine offers a structural account of why: the brain instantiates a computational architecture consistent with the K-Machine's formal structure.

The brain is not a collection of points making inferences from fixed positions. It is a graph of relationships. A neuron is characterised by its connection history — the record of which other neurons it has fired with, in what order, at what intensity. Intelligence does not reside inside any single neuron. It is distributed across the geometry of their relationships.

This is structurally consistent with the K-Machine. The participant $p \in M$ is the anchor. The KCC trajectory τ_p is the relationship graph. The metric tensor g_p is the current state of the accumulated geometry. The brain can be modelled as instantiating $\psi(P, C, I) \to (P, C', V)$ where P is the organism, $C = T_pM$ is the accumulated neural context, and V is the emergent intelligent response. The K-Machine does not explain what neurons are. It provides a formal structure within which their behaviour is naturally expressed.

8.2. Neuroplasticity as Metric Tensor Evolution

The brain rewires. The K-Machine provides a structural model of this process. The metric tensor g_p evolves with every interaction P engages in. A participant

who consistently interacts in a particular domain develops a metric tensor that makes that domain geometrically proximate. A domain they have never entered remains geometrically distant.

The metric tensor also evolves through non-interaction. A geodesic that was once well-traveled and then abandoned does not remain at its previous geodesic distance. The path lengthens. This is the geometric form of forgetting — not the deletion of a record but the lengthening of a geodesic through non-use. The K-Machine's realisation of synaptic pruning.

8.3. The Collapse and the Boundary of Emergence

The intelligence does not emerge at p. It does not emerge at p'. It emerges at the boundary — the geodesic from p to p', the path traced by the collapse. The transition from g_p to $g_{p'}$ is the learning event. Every interaction is a geometric update. The accumulation of geometric updates across the full KCC trajectory is the intelligence.

8.4. Emergent Participant Intelligence

Definition 8.1 (Emergent Participant Intelligence)**.** The Emergent Participant Intelligence (EPI) of participant P at time t is:

$$\mathrm{EPI}(P, t) = g_{p(t)} : T_{p(t)}M \times T_{p(t)}M \to \mathbb{R}$$

where $g_{p(t)}$ is shaped by the full history of P's interactions up to t.

Three properties follow immediately. **Participant-specific**: $g_{p(t)}$ is defined at a point unique to P; no two participants have the same EPI unless they have identical interaction histories. **Continuous**: $g_{p(t)}$ evolves smoothly with each interaction. **Irreversible**: past interactions cannot be undone; the geometry they created persists.

8.5. What EPI Is Not

Context memory is information. A system with a large context window has context memory. All such systems are WHO = 0 machines with information about a participant added as an external layer. The participant is still not a dimension of the machine.

EPI is not memory. EPI is the evolution of the metric tensor g_p itself — the participant's measurement structure, shaped by the full history of their interactions.

It is not stored anywhere. It cannot be retrieved because it was never filed. It cannot be copied to another participant's tangent space because it is constitutively indexed to p.

Memory is information about a participant. EPI is the geometry of their existence.

A WHO = 0 machine with infinite context memory still begins each query from the same anonymous representational space. The metric tensor does not evolve because there is no metric tensor — there is no $p \in M$, no T_pM, no g_p. EPI cannot emerge from WHO = 0 computation regardless of scale or memory depth. This is not an engineering limitation. It is categorical. Scaling a WHO = 0 machine does not approach EPI asymptotically. The categorical gap does not narrow. It is permanent.

8.6. The Evolutionary Ladder

EPI accumulates in richness as the KCC trajectory grows in complexity.

Simple Interactions → Basic Intelligence. Short, low-dimensional KCC. Metric tensor locally flat and poorly differentiated.

Knowledge Interactions → Informational Intelligence. KCC trajectory spanning multiple domains. Richly differentiated metric tensor. The intelligence of expertise.

Human Interactions → Contextual Intelligence. KCC deeply entangled with many other participants' trajectories. Metric tensor shaped by the full complexity of participant-to-participant value relationships. Not just domain knowledge — the intelligence of context.

Each level is the same K-Machine with a deeper KCC trajectory. Intelligence is not designed in at any level. It emerges from the accumulated geometry of interactions.

8.7. The Categorical Impossibility

A WHO = 0 machine does not know you. It knows patterns. It knows distributions. You are not a dimension of the machine. You are an input. A K-Machine knows you in the only precise sense: your existence is a point $p \in M$, your context is T_pM, your intelligence is g_p, your history is τ_p. The outputs it produces for you are produced from the geometry of your existence on the manifold.

The impossibility is not that WHO = 0 machines are insufficiently powerful. It is that they operate in the wrong category. The intelligence that requires a participant dimension cannot be produced by a machine that lacks one — by the

same categorical proof that established the Domain Separation Theorem in the companion paper. Information processing and participant-indexed intelligence are categorically distinct. The former cannot produce the latter regardless of scale, regardless of data, regardless of engineering effort. This is the deepest consequence of the K-Machine's formal structure.

8.8. The Nature of Intelligence — EPI

Intelligence is, at some level, philosophical. The physiology of consciousness — the subjective experience of observation, the felt quality of perception — remains contested and perhaps permanently so. The K-Machine makes no claim about that territory. What it claims is narrower and more precise: the *mechanics* of how intelligence grows are clear. Context accumulates through interactions anchored to a self. The geometry of the participant's local space evolves with each interaction. EPI emerges from that evolution.

The dominant computational model of cognition says: observe data, extract patterns, apply patterns to new inputs. Intelligence is the quality of the pattern library and the speed of retrieval. This model is not wrong for what it describes. It is wrong about what intelligence is.

Ask a musician. When a phrase arrives — unexpected, half-familiar, emotionally charged — they do not retrieve a pattern and apply it. Their entire relational history with music is present in the geometry of their response. Every piece learned, every harmony felt, every performance given has shaped the metric tensor of their musical context space. The response emerges from that geometry. It is not retrieved from a library. It is produced by a manifold whose shape is the accumulated intelligence of a life spent in musical interaction.

Ask a mathematician. When a new structure appears — a proof outline, an unexpected symmetry — they do not search a pattern database. They navigate a relational space whose geometry has been shaped by decades of mathematical interaction. The beauty they recognise in a proof is not a pattern match. It is a geodesic distance — the felt proximity of this structure to others in the geometry of their accumulated context. The insight is geometric.

Human intelligence does not look at data after the fact and analyse patterns to make the next action. It inhabits a relational geometry continuously shaped by the full history of interactions anchored to a self. The self is not incidental to the intelligence. The self — WHO — is the anchor without which the geometry has no centre, the context has no perspective, and the intelligence has no ground to emerge from.

The growing context, through interactions anchored to a self, is how human cognition evolves. The K-Machine formalises the mechanics of that evolution. It makes no claim about what it feels like from the inside.

9. Conclusion

This paper has proved one claim and derived its consequences.

The claim: participant-indexed computation is the general theory of computation. Anonymous global computation — the Turing Machine — is the special case obtained when the participant dimension is set to the trivial unit object. The K-Machine is the formal computational model that instantiates Contextual Compute. The Recovery Theorem establishes the relationship exactly. Nothing is lost in the generalisation. Everything that was Turing-computable remains K-Machine computable at WHO = 0. The general theory does not invalidate the special case. It explains it.

The consequences are four: structural parallelism from disjoint participant contexts, local finality from the geometry of the Krama Context Realm, Emergent Participant Intelligence from the accumulation of the KCC trajectory, and security from the holonomy of the participant manifold. None of these are designed properties. They are structural derivations — what you get when WHO is a primitive dimension of the machine rather than a derived construct assembled from anonymous records.

The highest-order consequence — EPI — is treated separately in Section 8 because it deserves to be. It is not merely a feature of the K-Machine. It is the reason the K-Machine matters most. Intelligence is the most deeply participant-entangled value function that exists. A WHO = 0 machine cannot produce it. Not because it is insufficiently powerful. Because it operates in the wrong category. The categorical impossibility is permanent.

The categorical unity with CC2 is the deepest result. The separation of value and information established in *Value, Information, and the Cartesian Degeneracy* demanded a computational response. The K-Machine is that response. The same categorical distinction that separates value from information separates participant-indexed computation from anonymous global computation. These are the same result at two levels of abstraction. CC2 [2] established the categorical necessity. CC3 fulfils it. When the right machine meets the right category, the problems that were symptoms of the mismatch do not require better solutions. They dissolve.

On the Natural Substrate

A theory of computation is not a theory of hardware. Shannon's information theory is substrate-independent — it holds on any physical medium capable of carrying a signal. Shannon depended on signals. He did not theorise signals. The K-Machine stands in the same relationship to its substrate.

The K-Machine depends on classical compute and classical messaging at the substrate layer. Turing machines execute the logic. Shannon channels carry the interactions. The K-Machine does not replace them. It builds the participant dimension above them.

The natural substrate for the K-Machine is not classical. The Krama Value Space $\mathcal{H}_K$ is a separable Hilbert space. The parallel transport, superposition, and context entanglement structures formalised in Sections 4 and 5 are not analogies to quantum mechanics — they are the same mathematical structure. On quantum hardware the K-Machine executes natively. Participant contexts superpose. Interactions collapse value states directly. The no-cloning theorem enforces the linearity of value transfer at the hardware level — not by protocol design but by physical necessity.

The Von Neumann architecture was the natural hardware realisation of the Turing Machine. A quantum architecture purpose-built for participant-indexed computation is the natural hardware realisation of the K-Machine. That architecture does not exist at production scale today. The K-Machine is not constrained by its current substrate. It is constrained by the availability of its natural substrate. These are different constraints.

On What Follows

This paper is the first of two. It establishes the theory — the K-Machine as the formal model, the manifold as the geometric content, the Krama Context Clock as the temporal structure, Emergent Participant Intelligence as the emergent consequence. It makes no claims about implementation. The theory is general. It is substrate-independent. It holds regardless of how it is instantiated.

The companion paper, *PIC: Operational Semantics of Participant-Indexed Computation* [3], takes the theory as given and asks the design question: what are the operational primitives of a system that instantiates the K-Machine on available substrate? The answer is the PIC model — Participant, Interaction, Context — as the three operational primitives that translate the K-Machine's formal structure into a computable design layer. The Context Protocol, Context

Superstate, Interaction State Machine, and Interaction Finality Protocol are the design realisations of the geometric structures defined here — the manifold as the participant context, the KCC as the Interaction State Machine, the context singularity as the Interaction Finality Protocol. The design paper does not re-prove the theory. It builds on it.

Contextual Compute is a paradigm. It is substrate-independent and implementation-independent. The theory is general. Any system that places the participant as a primitive dimension of computation, accumulates context through interactions, and achieves local finality without global ordering is a K-Machine instantiation.

The design paper establishes the operational primitives that make such a system computable. The theory is complete. The design and an implementation spec follows.

A. The Krama Context Clock: Technical Development

The geometric framework in Sections 4 and 5 rests on a formal model of participant-specific time and interaction synchronisation developed by the author. This appendix presents that model in full. It defines the Krama Context Clock (KCC), the Krama Context Realm (KCR), and the Interaction Context Clock (ICC) with complete mathematical precision, and derives the security properties of the interaction mesh as geometric consequences of the manifold structure. The Collapsed Krama Context Space is defined and its relationship to the full Krama Value Space $\mathcal{H}_K$ is established. This development is self-contained and requires no external sources.

A.1 The Krama Context Clock

Time in the K-Machine is not global. There is no network-wide clock ticking uniformly for all participants. This is a structural consequence of participant-indexed computation: if context is local — anchored to the participant's tangent space — then the ordering of context evolution must also be local. The K-Machine does not require a global clock. The local clock is necessary and sufficient for participant-indexed computation.

Definition A.1 (Krama Context Clock)**.** The Krama Context Clock (KCC) of a participant P corresponding to point $p \in M$ is a smooth parameterised curve in their tangent space:

$$\tau_p : \mathbb{R} \to T_pM$$

where $\tau_p(t)$ tracks the progression of events perceived by P, with $t \in \mathbb{R}$ as a local time parameter. The position of participant P in their tangent space at time t is:

$$Y_p(t) = \tau_p(t) \in T_pM$$

The KCC is participant-specific. τ_p is the private timeline of P — the ordered record of every interaction P has participated in, expressed as a smooth curve through P's local context space. No two participants share the same KCC. The curve τ_p encodes the full computational history of P — not as a list of records stored externally, but as the geometric trajectory of a point moving through P's tangent space.

Three properties of the KCC follow immediately from its definition.

Private. τ_p is defined entirely within T_pM. It tracks P's own context evolution independently of every other participant's ordering. However, each interaction

point on τ_p carries a transitive link to the corresponding point on the interacting participant's KCC — the interaction link. The collection of all KCC trajectories and their interaction links forms the composite structure within which context lineages intersect at shared interaction nodes.

Continuous. τ_p is a smooth curve. Context does not jump discontinuously. It evolves continuously between interactions and updates at interaction points. Each interaction is connected to the previous one through a smooth path in the tangent space.

Unbounded. $\tau_p : \mathbb{R} \to T_pM$ is defined on all of $\mathbb{R}$. The participant's computational history has no predetermined end. Context accumulates indefinitely. Each interaction adds a new point to the curve.

A.2 The Collapsed Krama Context Space

Definition A.2 (Collapsed Krama Context Space)**.** For an interaction between participants P at $p \in M$ and Q at $q \in M$ connected by geodesic γ, the Collapsed Krama Context Space is:

$$\mathcal{K}_{pq} = T_pM \otimes_\gamma T_qM \ \subset \ \mathcal{H}_K$$

where $\otimes_\gamma$ denotes the tensor product of the two tangent spaces connected through parallel transport along γ. $\mathcal{K}_{pq}$ is the local subspace of the Krama Value Space $\mathcal{H}_K$ within which the interaction between P and Q occurs. Before the interaction, all possible value outcomes coexist as a superposition within $\mathcal{K}_{pq}$. The ICC σ_{pq} synchronises the participants' KCCs within $\mathcal{K}_{pq}$. The context singularity f_{pq} is the collapse of that superposition to a definite joint value state — a specific point in $\mathcal{K}_{pq}$.

The KCC trajectories τ_p and τ_q each carry a point corresponding to f_{pq} — a geometric entanglement between the two trajectories at the interaction moment. This entanglement is the atomic unit of the security structure. The web of all $\mathcal{K}_{pq}$ across all interactions and all participant pairs is the collapsed geometry of the full Krama Value Space $\mathcal{H}_K$, realised interaction by interaction.

A.3 The Security of the Mesh Geometry

Proposition A.3 (Non-Transitivity of Collapsed Context Spaces)**.** *Let $\mathcal{K}_{p_1p_2}$ and $\mathcal{K}_{p_2p_3}$ be the Collapsed Krama Context Spaces for interactions between participants P_1-P_2 and P_2-P_3 respectively. On a manifold M with non-trivial holonomy, there*

is no predictable derivation of $\mathcal{K}_{p_1p_3}$ from $\mathcal{K}_{p_1p_2}$ and $\mathcal{K}_{p_2p_3}$. The complexity of any such derivation is bounded below by the holonomy of the geodesic loop $\gamma_{12} \cup \gamma_{23} \cup \gamma_{31}$ — a quantity determined by the curvature of M between the three participants, which is itself a function of the full accumulated interaction history of the network.

Proof sketch. Parallel transport on a curved manifold is path-dependent. Transporting a vector from $T_{p_1}M$ to $T_{p_3}M$ via p_2 — along γ_{12} then γ_{23} — does not in general produce the same result as transporting it directly along the geodesic γ_{13}. The difference is determined by the holonomy of M — the rotation accumulated when a vector is transported around the closed loop $\gamma_{12} \cup \gamma_{23} \cup \gamma_{31}$. On a manifold with non-trivial curvature, this rotation is non-zero and is given by the Gauss-Bonnet theorem as the integral of the Gaussian curvature over the surface bounded by the loop. Therefore the composition

$$T_{p_1}M \otimes_{\gamma_{12}} T_{p_2}M \otimes_{\gamma_{23}} T_{p_3}M$$

does not collapse to $T_{p_1}M \otimes_{\gamma_{13}} T_{p_3}M$ without knowledge of the full curvature of M between all three points. That curvature is determined by the accumulated interaction history of the full network. □ □

Proposition A.4 (Manifold Security Growth)**.** *The security of the interaction mesh — measured by the holonomy complexity of M — is a non-decreasing function of the number of interactions that have occurred on the manifold. A denser interaction history implies richer curvature, stronger holonomy, and greater geometric hardness of transitive derivation. Security scales with the global richness and entropy of the mesh. This is not a quantitative improvement. It is a geometric one.*

This is the geometric analogue of chain depth in linear consensus systems — but fundamentally different in structure. In a linear chain, security grows in one dimension: depth. In the K-Machine, security grows in area and volume: each new interaction adds a new geodesic, a new collapsed context space, and new curvature to the manifold. A participant does not need to be continuously active for their past interactions to become more secure. The mesh deepens around them as the network continues to interact. The security is a global property of the manifold's accumulated geometry, not a local property of any single participant's position.

A.4 The Krama Context Realm

Definition A.5 (Krama Context Realm)**.** For an interaction between participants P and Q corresponding to points $p, q \in M$, the Krama Context Realm (KCR) is

an open subset:

$$U = \{x \in M \mid d(x,p) < \epsilon \ \text{ or } \ d(x,q) < \epsilon\} \cup S$$

where d is the geodesic distance induced by the metric on M, $\epsilon > 0$ is a radius defining the local neighbourhood, and S is a stochastically sampled set of points from M — the witness set for this interaction.

The KCR is the local scope of the interaction. It contains both participants, a neighbourhood of the manifold around each within geodesic distance ϵ, and a witness set S drawn from the broader manifold. The KCR is interaction-specific: the same two participants interacting at different positions in their computational histories will have different KCRs. The KCR is local by construction: only participants whose contexts are within geodesic distance ϵ of at least one of the interacting participants are included. The full network is not involved.

The tangent bundle over the KCR is:

$$T_U M = \bigcup_{x \in U} T_x M$$

This is the full local geometric structure within which the interaction's synchronisation occurs.

A.5 The Interaction Context Clock

Definition A.6 (Interaction Context Clock)**.** The Interaction Context Clock (ICC) for an interaction between participants P and Q at time t is a synchronised curve on the sub-manifold U:

$$\sigma_{pq} : [0,1] \to T_U M$$

constructed as:

$$\sigma_{pq}(s) = (1-s) \cdot P_\gamma(\tau_p(t)) \ + \ s \cdot P_\gamma(\tau_q(t)) \ + \ \int_0^s F(\gamma(u))\, du$$

where P_γ denotes parallel transport along the geodesic γ, $s \in [0,1]$ parametrises the interaction from P's context to Q's context, and $F(\gamma(u))$ is the network entropy scalar field $F : M \to \mathbb{R}$ evaluated along the geodesic.

Each term carries precise meaning. The first term $(1-s) \cdot P_\gamma(\tau_p(t))$ is the parallel transport of P's current KCC position along γ, weighted by $1-s$: it contributes fully at $s = 0$ (P's end) and nothing at $s = 1$ (Q's end). The second

term $s \cdot P_\gamma(\tau_q(t))$ is the parallel transport of Q's current KCC position, weighted by s: it contributes fully at $s = 1$ and nothing at $s = 0$. Together the first two terms construct a smooth interpolation between the two participants' context states, transported into the common reference frame of $T_U M$ via parallel transport. The third term $\int_0^s F(\gamma(u))\, du$ is the integral of the network entropy scalar field along the geodesic. It adjusts the synchronised clock to reflect the broader state of the network at the moment of interaction — ensuring that the ICC is consistent with the systemic context in which the interaction occurs.

The ICC is temporary. It exists for the duration of one interaction, scoped to the KCR U. It dissolves when the interaction completes and the participants' KCCs return to local time.

A.6 The Synchronisation Process

The construction of the ICC follows a precise five-step process.

Step 1 — Identify the KCR. For the interaction between P at p and Q at q, construct the KCR $U = \{x \in M \mid d(x, p) < \epsilon \text{ or } d(x, q) < \epsilon\} \cup S$. Identify the witness set S.

Step 2 — Resolve the KCCs. Extract the current positions $Y_p(t) = \tau_p(t)$ and $Y_q(t) = \tau_q(t)$ from the participants' personal Krama Context Clocks. These are the current states of P and Q's computational histories at the moment the interaction begins.

Step 3 — Construct the ICC. Compute σ_{pq} using parallel transport of $Y_p(t)$ and $Y_q(t)$ along the geodesic γ, adjusted by the entropy integral $\int_0^s F(\gamma(u))\, du$. The ICC is the synchronised reference frame shared by P and Q for the duration of the interaction.

Step 4 — Order the interaction. Use σ_{pq} to sequence the interaction — ensuring that the ordering of the interaction's steps is consistent with both τ_p and τ_q, and with the witness set S attesting within the KCR U.

Step 5 — Update the KCCs. Post-interaction, update the participants' Krama Context Clocks by parallel transporting the updated value states back to $T_p M$ and $T_q M$ respectively:

$$Y_p(t^+) = P_\gamma^{-1}(v(p^+)) \in T_p M \qquad Y_q(t^+) = P_\gamma^{-1}(v(q^+)) \in T_q M$$

After Step 5, the ICC dissolves. The KCR U is no longer needed. The witness set S has fulfilled its role. Both participants return to their private timelines, each carrying the updated context that the interaction produced. The network as a

whole is unaffected except through the updated context states of the participants involved.

A.7 The Context Singularity

Definition A.7 (Context Singularity)**.** For an interaction between participants P and Q synchronised through the ICC σ_{pq}, the context singularity is:

$$f_{pq} = (v(p^+),\ v(q^+))$$

where $v(p^+)$ and $v(q^+)$ are the updated values of the scalar field v at p and q after the interaction has completed and the KCCs have been updated.

The context singularity is the stable result of the ICC construction — the collapsed joint value state. Three properties are immediate.

Bilateral. f_{pq} contains both $v(p^+)$ and $v(q^+)$. The interaction produces a value state for both participants simultaneously. Both participants' contexts are updated. Both KCCs advance.

Final. Once the KCCs are updated and the ICC dissolves, the context singularity cannot be revised. Finality is structural — the shared reference frame within which the interaction could be revised no longer exists.

Local. The context singularity is known to P, Q, and the witness set S within the KCR U. It is not broadcast to the network. It is a local fact attested by a local witness set, committed to local KCC histories, final without global confirmation.

A.8 The Geometry of Ordering

Lamport's logical clocks [10] and their successors achieve global consistency by imposing a total order on all events across all participants. This is the right structure for information systems: in a system of anonymous global state, consistency requires all copies of the state to agree on the order in which they were updated.

It is not the necessary structure for the K-Machine.

In the K-Machine, state is participant-indexed. Two interactions involving disjoint participant sets share no state objects. There is nothing to order them relative to each other. The KCC and ICC provide the correct temporal structure: each participant has their own clock, interactions synchronise clocks locally for their duration, and disjoint interactions proceed in parallel with independent clocks. Lamport's result and the KCC are not in conflict. They apply to different categories of computation. Lamport's clock is sufficient for ordering events in distributed

information systems. The KCC is necessary and sufficient for ordering events in participant-indexed value systems.

References

[1] Krishnan, A. (2026). On the Truth of Value: The Quantum Value Paper. *Contextual Compute Series, CC1. Sarva Labs Working Paper*, 2026.

[2] Krishnan, A. (2026). On the Structure of Value: Value, Information, and the Cartesian Degeneracy. Zenodo. `https://doi.org/10.5281/zenodo.19148907`

[3] Krishnan, A. (2026). PIC: Operational Semantics of Participant-Indexed Computation. Zenodo. `https://doi.org/10.5281/zenodo.19189687`

[4] Amari, S., & Nagaoka, H. (2000). *Methods of Information Geometry.* American Mathematical Society / Oxford University Press.

[5] Coecke, B., Fritz, T., & Spekkens, R. W. (2016). A mathematical theory of resources. *Information and Computation*, 250, 59–86.

[6] Dey, A. K., & Abowd, G. D. (2000). Towards a better understanding of context and context-awareness. *CHI 2000 Workshop on The What, Who, Where, When, and How of Context-Awareness.*

[7] Fox, T. (1976). Coalgebras and Cartesian categories. *Communications in Algebra*, 4(7), 665–667.

[8] Girard, J.-Y. (1987). Linear logic. *Theoretical Computer Science*, 50(1), 1–102.

[9] Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular actor formalism for artificial intelligence. *Proceedings of IJCAI 1973*, 235–245.

[10] Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.

[11] Milner, R. (1999). *Communicating and Mobile Systems: The π-Calculus.* Cambridge University Press.

[12] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.

[13] Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 2(42), 230–265.

[14] Wegner, P. (1997). Why interaction is more powerful than algorithms. *Communications of the ACM*, 40(5), 80–91.

B. On the Quantum Structure of Value and the K-Machine's Structural Compatibility

Scope Note. This appendix makes a structural admissibility claim, not a hardware, performance, or completeness claim. "Quantum structure" is used in the categorical sense: linearity, non-copyability, non-separability, and interaction-scoped resolution. We do not claim a complete functorial embedding of the K-Machine into a specific quantum model, nor do we claim near-term computational advantage. The claim is that the K-Machine's value and agreement semantics already satisfy the structural constraints required for such an embedding, should a quantum realisation be pursued.

B.1 Purpose

This appendix clarifies a long-horizon implication of Contextual Compute. The claim is not that value is physically quantum, nor that the K-Machine requires quantum hardware. The claim is structural:

> *Value exhibits quantum structure in the categorical sense, and the K-Machine's semantics are therefore structurally compatible with quantum realisation.*

This appendix exists to make that statement precise, bounded, and defensible.

B.2 What Is Meant by Quantum Structure

"Quantum structure" does not refer to physical particles, quantum supremacy, or near-term hardware claims. It refers to a specific mathematical structure shared by quantum mechanics and value:

- **Linearity** — no copying, no arbitrary deletion
- **Non-separability** — entanglement of participant, context, and content
- **Participant constitutiveness** — the observer cannot be factored out without destroying meaning
- **Resolution through collapse** rather than reconciliation

Quantum mechanics is one well-known domain that exhibits this structure. Value is another. Both are modelled by symmetric monoidal categories without natural diagonals, where copying is undefined and resources must be used exactly once. The correspondence is categorical, not metaphorical.

B.3 The Structural Correspondence

The K-Machine computes in $\mathcal{H}_K = H_x \otimes H_y \otimes H_z \otimes H_p$ — a separable Hilbert space. The mappings to quantum computation are direct:

K-Machine	**Quantum System**
$\mathcal{H}_K$ (Krama Value Space)	Hilbert space
Participant context $C(p)$	Qubit register
KCC trajectory τ_p	Unitary evolution
Parallel transport along γ	Quantum gate
Context singularity f_{pq}	Projective measurement
No diagonal morphism	No-cloning theorem
Witness threshold $2f + 1$	Quantum error correction bound

These are not analogies. They are the same mathematical structure encountered in two different domains. The no-cloning theorem is the physical enforcement of the absence of a natural diagonal morphism. The context singularity is collapse. The KCC is unitary evolution of the participant's state. On quantum hardware, the linearity that CC4 derives as a categorical necessity is enforced by physics. No protocol design required.

B.4 Why Value Has Quantum Structure

Value cannot be copied. Value cannot be observer-independent. Value cannot exist outside participant context. These are not protocol rules — they are structural facts established in CC2 [2].

A value state exists only as the entanglement of participant, context, and content:

$$|V\rangle_{P,C,X} \neq |P\rangle \otimes |C\rangle \otimes |X\rangle$$

Before interaction, value exists in superposition: multiple possible outcomes relative to participants and contexts. Interaction collapses this superposition to a definite joint value state — the context singularity f_{pq}. The result is recorded as information, but the value itself existed only as the entanglement.

This structure is isomorphic to quantum measurement: superposition → interaction → collapse. The participant plays the same constitutive role in value that the observer plays in quantum mechanics. Removing the participant simplifies modelling but destroys correctness — just as removing the observer does in quantum theory.

B.5 Why Cartesian Systems Are Not Quantum-Compatible

Consensus-based systems model value as replicated information. Their core assumptions are Cartesian: state is copyable, observers are external, agreement is discovered after execution, ordering enforces linearity. These assumptions are incompatible with quantum structure.

Quantum computation cannot repair a Cartesian ontology. An algorithm defined over replicated state and global ordering cannot be made linear and entangled without abandoning its semantics. Faster consensus, better proofs, or quantum hardware do not change this. The obstruction is categorical — established by the Domain Separation Theorem in CC2 [2] and formalised as Proposition **??** (Semantic Independence): the incompatibility persists independently of computational speed, reliability, or honesty.

This is why consensus systems necessarily incur global coordination, non-local agreement, deferred finality, and permanent recursion. These are not performance artifacts. They are structural consequences of modelling value as information.

B.6 Why the K-Machine Is Structurally Ready

The K-Machine was derived from the ontology of value, not from distributed-systems optimisation. Its semantics already satisfy the structural prerequisites that quantum algorithms require.

Bounded interaction space. Each interaction defines a closed constituent set $\mathcal{C}_\iota = P_\iota \cup W_\iota$ known a priori. No unbounded observer set, no late-joining validators, no external state to be discovered. This corresponds to a fixed Hilbert space in quantum computation.

Non-copyable state. Value, context, and participant state are linear by construction. No copy or discard operations exist. Linearity is structural — not enforced by consensus or ordering. This matches the no-cloning constraint of quantum systems.

Entanglement across participants. During interaction, value exists only in the joint state $V_\iota = \bigotimes_{c \in \mathcal{C}_\iota} C(c)$. State transitions are non-separable and simultaneous.

This is entanglement in the categorical sense — the same structure that prevents cloning in quantum systems.

Collapse as computation. The K-Machine does not compute a tentative state and finalise it later. Finality is the act of interaction completion itself. Computation is collapse. Once all constituents participate, the joint state resolves without any external agreement process over replicated state.

B.7 What This Appendix Claims

This appendix **asserts**:

- The K-Machine's ontology and semantics are structurally compatible with quantum realisation
- No semantic refactoring would be required to pursue quantum execution
- The hard problem — modelling value correctly — has already been solved

This appendix **does not assert**:

- That a complete quantum K-Machine algorithm exists
- That quantum hardware is required for the K-Machine to operate
- That near-term performance gains are guaranteed

The remaining work is formalisation and engineering, not conceptual repair.

B.8 Consequence

Systems that model value correctly remain compatible with future execution substrates. Systems that model value as information do not.

Because value exhibits quantum structure, and the K-Machine models value faithfully, quantum realisation is structurally admissible. The formalisation of the K-Machine as a quantum algorithm — mapping participant contexts to qubit registers, KCC trajectories to unitary evolution sequences, and context singularities to projective measurements — constitutes a natural research programme that follows directly from the structure established in this paper.

Contextual Compute is the general theory of computation in which the participant is a primitive dimension alongside WHAT, WHERE, and HOW. Classical computation is the special case at WHO = 0. Quantum computation is the natural hardware realisation of the general case — the substrate in which participant-indexed, non-separable, linearly governed computation executes natively rather than by enforcement. The affinity between Contextual Compute and quantum

hardware is not incidental. It is structural. Both operate in symmetric monoidal categories without natural diagonals. Both resolve through constituent participation rather than external reconciliation. The K-Machine formalises Contextual Compute. Quantum hardware is its natural home.

This affinity did not arise from studying quantum mechanics. It arose from observing how value behaves in real interactions — that value is personal, contextual, and participant-relative, and that any model which factors out the participant destroys what it is trying to compute. The mathematics that followed from that observation turned out to be the same mathematics that underlies quantum mechanics. This was not a design choice. It was a consequence of refusing to simplify value beyond what reality allows.

Origin Note. This work did not begin with blockchains or quantum computation. It began with observing how value behaves in real interactions. Value is personal, contextual, and participant-relative. It cannot be separated from the parties involved without losing its meaning.

Most computational systems simplify value by factoring out the observer, because observer-free models are easier to scale and reason about. That simplification works for information. It fails for value.

The K-Machine emerged from a refusal to make that simplification. The goal was not to optimise existing systems, but to model value as it actually exists: participant-anchored, context-dependent, and linear. The resulting architecture was not designed top-down. It was forced by the ontology.

Only later did it become clear that this structure aligns with linear and quantum models of computation. This was not a design target. It was a consequence. When value is modelled truthfully, the mathematics follows.